手撕 AI 题详解 —— Python 版（零基础也能写出来）

> **说明**：本文档为对外分享版，前雇主项目的量化指标已做量级化处理，方法论与话术保持原样。
> 定位：**算法题（力扣）继续用 C++**（见另一份《手撕AI题详解_C++版》第 3 章），**AI 题（softmax / MHA / kmeans / 决策树）用 Python**——这份文档专门解决"Python 写不出来"。
> 全部代码已实测：**39 项断言全部通过**，其中 torch 部分装了 CPU 版 torch 2.14 真跑过，包括与官方 `F.scaled_dot_product_attention`、`F.cross_entropy` 的逐元素对比。配套可运行文件：`ai_interview_py.py`（同目录），`python3 ai_interview_py.py`。
> 阅读顺序：第 1、2 章（语法地基，1 小时）→ 第 3 章逐题（主体）→ 第 4 章 torch → 第 5 章验证法。

---

0. 先给你一个心理预期

你说"Python 基础不好"，但**手撕 AI 题需要的 Python 只有 30 个写法**，而且全是"数学公式直译"。真正的门槛不是语法，是三件事：

真正的门槛	为什么 C++ 选手会卡	本文怎么解决
numpy 的"轴"概念（axis / keepdims）	C++ 里你写循环很自由，numpy 你要先想"沿哪一维聚合"	第 2.2 节用一张"循环 ↔ axis"对照表讲透
广播规则（shape 怎么对齐）	C++ 没有这概念，写错就报 shape 不匹配	第 2.3 节给三条规则 + `None` 插轴的口诀
"一行顶你二十行"的 API（argmin / where / bincount / gather）	你不知道有这个函数，只能硬写循环	第 2.5 节列 12 个必会 API，每个配 C++ 等价写法

一句话心法：C++ 里你写的是"过程"，Python/numpy 里你先想"我要的结果是什么形状"，再选一个能产出这个形状的函数。**每次写代码前，先在纸上写 shape**。

---

1. Python 语法速成（只讲手撕用得到的）

> 目标：让你能读懂并写出第 3 章的所有代码。全部用 C++ 对照，跳过装饰器、生成器、异步这些用不到的。

1.1 十条语法对照表（必背）

要做的事	C++	Python	备注
定义函数	`int f(int x) { return x+1; }`	`def f(x): return x + 1`	**没有类型声明**，返回值靠 `return`
定义类	`class A { public: int x; };`	`class A:` + `__init__`	`def __init__(self, x): self.x = x`
成员变量	`this->x`	`self.x`	方法第一个参数永远是 `self`
循环	`for (int i = 0; i < n; ++i)`	`for i in range(n):`	`range(n)` = 0..n-1
遍历元素	`for (auto& v : a)`	`for v in a:`	要下标用 `enumerate(a)` → `for i, v in enumerate(a):`
数组	`vector<int> a(n, 0);`	`a = [0] * n`	或用 numpy（见第 2 章）
长度	`a.size()`	`len(a)`	Python 的 `len` 对 str/list/ndarray 都适用
条件	`x > 0 ? a : b`	`a if x > 0 else b`	或 `np.where`（数组版）
整数除法	`7 / 2 == 3`	`7 // 2 == 3`	**`/` 在 Python 里是浮点除法**，`7/2=3.5`
幂	`pow(x, 2)` / `x*x`	`x ** 2`	

1.2 五个"语法不同但一定用到"的点

```
# 1) 缩进就是作用域（没有大括号！）。一个 tab 或 4 个空格，全文件保持一致。
def f(x):
    if x > 0:
        return 1        # 这行属于 if
    return 0            # 这行属于函数

# 2) 列表推导式：一行写完"循环 + 筛选 + 收集"
ious = [iou(boxes[i], boxes[j]) for j in rest]          # 等价 C++: for(...) v.push_back(...)
pos = [x for x in a if x > 0]                           # 带筛选

# 3) 布尔数组直接当下标用（numpy 的核心技巧，C++ 里没有）
X[mask]        # 取出所有 mask 为 True 的行
y[~mask]       # ~ 是布尔取反

# 4) 多重赋值 / 交换
a, b = b, a
lo, hi = 0, len(a)

# 5) 返回值可以是多个
def kmeans(...):
    return labels, centers      # 返回元组，调用方 `lab, cen = kmeans(...)`
```

1.3 你会看到但可能没写过的符号（第 3 章会反复出现）

符号	含义	例子
`@`	矩阵乘（Python 3.5+）	`Q @ K.T`
`.T`	转置	`K.T`
`:`	切片	`x[:, sl]` / `x[:-1]`
`None`（在索引里）	插入一个新轴	`X[:, None, :]` 见 2.3
`**2`	平方	`(p ** 2).sum()`
`-1`（在 axis 里）	"最后一维"	`x.sum(-1)`
`//`	整除	`D // h`
`~`	布尔取反（numpy）	`y[~mask]`
`np.inf` / `-1e9`	无穷 / 极小值（做 mask）	`np.where(mask, -1e9, scores)`

---

2. numpy 地基：读懂 Python 手撕代码的钥匙

> 这一章是本文档的核心。**看懂这五个小节，第 3 章的代码你会觉得比 C++ 还简单。**

2.1 ndarray 是什么

```
import numpy as np
x = np.array([[1., 2., 3.], [4., 5., 6.]])   # 2 行 3 列
x.shape      # (2, 3)   —— 元组，写法像 C++ 的 pair，用 x.shape[0] 取行数
x.dtype      # float64
```

关键区别：C++ 你管"一维 vector + 自己算下标"；numpy 里 shape 是数据结构的一部分，所有运算都按 shape 自动对齐。代价是：**shape 写错就报错**——这是好事，等于免费的类型检查。

2.2 axis 与 keepdims（第一个大坎）

一句话：`axis=k` 表示"**沿第 k 维做聚合，做完这一维就消失**"。

```
x = np.array([[1., 2., 3.],
              [4., 5., 6.]])        # shape (2,3)
x.sum(axis=0)   # [5, 7, 9]       沿"行"方向压 → 每列求和，shape 从 (2,3) 变 (3,)
```

```
x.sum(axis=1)   # [6, 15]         沿"列"方向压 → 每行求和，shape 变 (2,)
x.sum(axis=-1)  # [6, 15]         -1 = 最后一维，等价 axis=1（推荐用 -1，代码通用）
x.max(axis=1)   # [3, 6]          每行最大值
```

C++ 对照： `axis=1` 就是"外层循环遍历行、内层累加列"，但 numpy 把内层循环省掉了：

```
// C++ 求每行和
for (int i = 0; i < n; ++i) { float s = 0; for (int j = 0; j < m; ++j) s += x[i][j]; rowsum[i] = s; }
```

```
rowsum = x.sum(axis=1)          # 一行搞定
```

keepdims=True 是干什么的（softmax 里必用）：

```
x.sum(axis=1)                # shape (2,)     ← 少了一维，没法直接除回 (2,3)
x.sum(axis=1, keepdims=True)  # shape (2,1)    ← 保住维度，可广播到 (2,3)
x / x.sum(axis=1, keepdims=True)     # 每行归一化，正确
x / x.sum(axis=1)                    # 报错：(2,3) 除 (2,) 语义不明确
```

口诀：只要这个聚合结果后面还要参与逐元素运算（相减/相除），就必须 `keepdims=True`。

2.3 广播（第二个大坎）

规则三条，按顺序判断，满足就停： 1. 从**最后一维**往前比，维度相等 → 可以。 2. 某一维是 **1** → 自动扩展成对方长度（复制）。 3. 维度数不同 → 缺的那边在前面补 1。

```
a = np.zeros((3, 4))
b = np.zeros((   4))     # 补成 (1,4)，再扩成 (3,4)      ✅
c = np.zeros((3, 1))     # 扩成 (3,4)                    ✅
d = np.zeros((2, 4))     # 3 != 2 且都不是 1             ❌ 报错
```

`None` 插轴的用法（kmeans 里那句长代码就是它）：

```
X = np.zeros((5, 2))          # 5 个样本，每样本 2 维
X[:, None, :].shape           # (5, 1, 2)   ← 在中间插一个长度 1 的轴
C = np.zeros((3, 2))          # 3 个中心
C[None, :, :].shape           # (1, 3, 2)
X[:, None, :] - C[None, :, :] # (5,1,2) - (1,3,2) → (5,3,2)   广播成立!
```

再 `.sum(-1)` → `(5,3)` = 5 个点到 3 个中心的距离平方。**用 C++ 写这段是三重循环，numpy 三行。**

2.4 索引：切片、布尔索引、高级索引

```
x = np.arange(12).reshape(3, 4)
x[0]          # 第 0 行
x[:, 1]       # 第 1 列          ← 逗号左边是行、右边是列
x[1:3]        # 第 1、2 行       ← 切片左闭右开
x[:, 1:3]     # 第 1、2 列
x[:, -1]      # 最后一列

# 布尔索引（最常用）
mask = x > 5
x[mask]       # 取出所有 >5 的元素，变成长度 k 的一维数组
y[labels == 2]         # 取出标签为 2 的那些样本（C++ 里要写循环判断）
y[~mask]               # 取反

# 高级索引（交叉取"每行的第 target 个元素"，交叉熵里用）
logits[np.arange(3), [2, 0, 1]]   # 取出 logits[0,2], logits[1,0], logits[2,1]
```

2.5 12 个必会 API（每个都给 C++ 等价思路）

API	作用	C++ 里你会怎么写
`x.max(axis, keepdims)`	按轴最大值	循环比较
`x.sum(axis)` / `x.mean(axis)` / `x.var(axis)`	求和/均值/方差	累加再除
`np.exp(x)` / `np.sqrt(x)` / `np.log(x)`	逐元素数学	循环 + `expf`
`np.where(cond, a, b)`	逐元素三目	`for` + `if`

API	作用	C++ 里你会怎么写
`x.argmin(axis)` / `x.argmax(axis)`	**最小/最大值的位置**（不是值）	循环记下标
`np.argsort(x)` / `np.argsort(-x)`	排序后的**下标**（降序用负号）	`sort` + 下标数组
`np.bincount(y)`	统计 0..k 每个整数出现几次	`int cnt[k] = {0}`
`np.unique(x)`	去重并排序	`sort` + `erase(unique)`
`np.concatenate(list, axis)`	沿某维拼接	两次 `insert`
`np.stack(list, axis)`	堆成新维度	新建矩阵逐元素拷
`np.allclose(a, b, atol)`	浮点近似相等（自检用）	手写 `fabs(a-b)<eps`
`x.copy()`	**深拷贝**（Python 里赋值是引用！）	C++ 的 `=` 就是值拷贝

> **最容易踩的坑**：`a = b` 在 Python 里是**引用**，不是拷贝。需要独立数组就写 `b = a.copy()`。kmeans 里 `new_centers = centers.copy()` 就是为这个——不 copy 会把上一步的中心改掉。

---

3. 逐题详解（numpy 版）

> 每题六段：**题目 → 30 秒口述 → 公式 → 代码 → 逐行讲解（重点 Python/numpy 语法）→ 追问 Q&A → 最易错 3 点**。

3.1 softmax（3 行，但每行都要会讲）

面试官原话："用 Python 写个 softmax。"

30 秒口述：

> "三步：先沿最后一维减去每行最大值防溢出，再取 exp，最后除以每行的和。numpy 里就是 `x - x.max(keepdims=True)`、`np.exp`、`e / e.sum(keepdims=True)`。keepdims 不能省，否则形状对不上。"

代码：

```python
def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)     # ① 平移：每行减去该行最大值
    e = np.exp(x)                               # ② 逐元素取指数
    return e / e.sum(axis=axis, keepdims=True)  # ③ 每行归一化
```

逐行讲解（Python/numpy 语法是这个题的考点）：

写法	语法点	为什么必须这么写
`x.max(axis=-1, keepdims=True)`	`axis=-1` = 最后一维；`keepdims=True` 保住这一维为长度 1	结果 shape 是 (2,1) 而不是 (2,)，才能和 (2,3) 广播相减。**写成 `x.max(axis=1)` 会报 "operands could not be broadcast"**
`x - (...)`	numpy 的 `-` 是**逐元素**，且自动广播	不需要任何循环
`np.exp(x)`	逐元素 exp（不是 `math.exp`，那个只能处理标量）	`math.exp` 传入数组会 TypeError
`e / e.sum(axis=-1, keepdims=True)`	逐元素相除，同样靠广播	换成 `e / e.sum(axis=-1)[:, None]` 也等价（两种写法都要能认）

手写时的一个自检：`softmax(x).sum(-1)` 必须全是 1.0。我在 `ai_interview_py.py` 里就是这么断言的。

追问 Q&A：- **Q：为什么减 max？** A：`ex` 上溢成 inf，再除 inf 得 nan；分子分母同乘 `e^(-max)` 值不变。 - **Q：`axis=0` 和 `axis=1` 分别是什么？** A：`axis=1`（=最后一维）是"每个样本自己归一化"——softmax 永远用这个；`axis=0` 是"同一位置上跨样本归一化"，那是错的。 - **Q：形状是 1 维时怎么办？** A：`axis=-1` 自动适配，所以函数写成 `axis=-1` 最通用。 - **Q：温度 / mask？** A：把 `x` 换成 `x/T`；mask 位置置 `-1e9`（不要用 `-np.inf`，因为 `-inf - (-inf)` 会出 nan；全屏蔽行仍是 nan，工业

实现要单独处理）。 - **Q：softmax 和 log_softmax 区别？** A：log_softmax = `x - max - log(sum(exp(x-max)))`，在对数域直接算，交叉熵用它更稳。

最易错 3 点：忘了 `keepdims`；用 `math.exp`；`axis` 写成 0。

3.2 交叉熵（考你对"高级索引"和 logsumexp 的理解）

面试官原话："手写交叉熵，不用 nn.CrossEntropyLoss。"

30 秒口述：

> "loss 等于 logsumexp(每行 logits) 减去正确类的 logit，再对 batch 取均值。我不先算 softmax 再取 log，直接在对数域算，避免小概率下溢。取正确类 logit 用高级索引 `logits[np.arange(N), target]`。"

代码：

```python
def cross_entropy(logits, target, weight=None):
    m = logits.max(axis=1, keepdims=True)                    # [N,1]
    lse = m[:, 0] + np.log(np.exp(logits - m).sum(axis=1))   # [N] logsumexp
    nll = lse - logits[np.arange(len(target)), target]       # [N] 取正确类的 logit
    if weight is not None:
        w = weight[target]                                   # [N]
        return (nll * w).sum() / w.sum()
    return nll.mean()
```

逐行讲解：

- `m = logits.max(axis=1, keepdims=True)` → shape (N,1)。**为什么这里要 keepdims**：下一行要 `logits - m`，需要 (N,C) - (N,1) 才能广播。
- `m[:, 0]` → 把 (N,1) 取成 (N,)。**语法点**：逗号左边 `:` 表示所有行，右边 `0` 表示第 0 列。
- `np.exp(logits - m).sum(axis=1)` → 每行 exp 之和，(N,)。
- `np.log(...)` → 逐元素 log。
- **`logits[np.arange(len(target)), target]`** ← 本题唯一的"技巧点"：两个下标数组交叉索引，等价于 C++ 的 `cpp for (int i = 0; i < N; ++i) picked[i] = logits[i][target[i]];` 这是 numpy 高级索引的标准用法，**面试官会专门看你会不会这个**（不会的话只能写循环，也正确但显得生疏）。
- `weight[target]` → 用 target 去查权重，得到每个样本的权重 (N,)。
- `(nll * w).sum() / w.sum()` → 加权平均。C++ 里你要累加两个变量再除。

追问 Q&A：- **Q：batch 版和单样本版的关系？** A：单样本就是 N=1；`mean` 是 batch 平均，框架的 `reduction='mean'` 就是它。 - **Q：`ignore_index` 怎么做？** A：把 target 等于 ignore_index 的样本 mask 掉：`valid = target != ig; return (nll[valid]).mean()`。 - **Q：为什么梯度是 p - y？** A：对 logit 求导，softmax + CE 的雅可比与 CE 的导数抵消，得到极简形式——这也是为什么框架把 softmax 和 CE 合成一个算子（数值更稳、更快）。 - **Q：多标签怎么办？** A：用 sigmoid + BCE：`F.binary_cross_entropy_with_logits(logits, targets)`，逐元素算 `log(1+e^-z) - y·z`。

最易错 3 点：`m` 忘了 keepdims；用 `softmax` 再 `log`（小概率下溢成 -inf）；交叉索引写成 `logits[target]`（那是取第 target 行，形状全错）。

3.3 LayerNorm（考 axis 和广播的理解）

代码：

```python
def layer_norm(x, gamma, beta, eps=1e-5):
    mu = x.mean(axis=-1, keepdims=True)          # [..., 1]
    var = x.var(axis=-1, keepdims=True)          # [..., 1]   (除 D，与框架一致)
    return gamma * (x - mu) / np.sqrt(var + eps) + beta
```

逐行讲解：

写法	语法点
`x.mean(axis=-1, keepdims=True)`	对最后一维求均值且保持维度，才能与 x 广播
`x.var(axis=-1)`	**numpy 默认是总体方差（除 D）**，正好与框架一致；如果要无偏方差写 `ddof=1`
`gamma * (...) + beta`	gamma/beta 形状 (D,)，自动广播到 (..., D)，**不用手动 reshape**
`np.sqrt(var + eps)`	eps 在 sqrt **里面**

追问：LN vs BN（统计量维度）、RMSNorm（去掉均值）、Pre-LN vs Post-LN、BN 推理用滑动平均。细节见 C++ 版 2.3 节。

最易错：`axis=0`（那是 BN 的方向）；eps 放外面；用 `ddof=1`。

3.4 IoU + NMS（考 while 循环 + argsort + 布尔索引）

代码：

```python
def iou(box_a, box_b):
    ix1, iy1 = max(box_a[0], box_b[0]), max(box_a[1], box_b[1])
    ix2, iy2 = min(box_a[2], box_b[2]), min(box_a[3], box_b[3])
    iw, ih = max(0.0, ix2 - ix1), max(0.0, iy2 - iy1)       # clamp 到 0
    inter = iw * ih
    area_a = max(0.0, box_a[2] - box_a[0]) * max(0.0, box_a[3] - box_a[1])
    area_b = max(0.0, box_b[2] - box_b[0]) * max(0.0, box_b[3] - box_b[1])
    union = area_a + area_b - inter
    return inter / union if union > 0 else 0.0

def nms(boxes, scores, thr=0.5):
    order = np.argsort(-np.asarray(scores))        # 降序排列后的下标
    keep = []
    while order.size > 0:
        i = int(order[0])                          # 当前最高分
        keep.append(i)
        if order.size == 1:
            break
        rest = order[1:]
        ious = np.array([iou(boxes[i], boxes[j]) for j in rest])   # 列表推导式
        order = rest[ious <= thr]                  # 布尔索引筛选
    return keep
```

逐行讲解：

- `np.argsort(-scores)`：`argsort` 返回**下标**而不是值；加负号实现降序。**这句不会就写不出 NMS**。
- `while order.size > 0`：numpy 数组用 `.size`（元素个数）或 `len()`；`order[0]` 是第一个元素。
- `keep = []` + `keep.append(i)`：Python 的 list，等价 C++ 的 `vector<int>`。
- `[iou(boxes[i], boxes[j]) for j in rest]`：**列表推导式**，等价 C++ 的 `cpp vector<float> ious; for (int j : rest) ious.push_back(iou(boxes[i], boxes[j]));`
- `ious <= thr` 得到一个**布尔数组**，直接拿来索引 `rest` → 筛出保留的下标。这是 numpy 相对 C++ 最爽的地方。
- `int(order[0])`：numpy 的整数是 `np.int64`，转成 Python `int` 更安全（有些库不接受 np 类型）。

追问：复杂度、Soft-NMS、IoU-aware、DETR 去 NMS、GIoU/DIoU/CIoU（详见 C++ 版 2.4 节）。

最易错：忘 clamp（负面积）；`argsort` 忘了负号（变成从小到大）；忘 `union` 减 `inter`。

3.5 K-Means（numpy 广播的最佳练习题）

面试官原话："手写 kmeans。"

30 秒口述：

> "E 步算每个点到每个中心的距离平方取 argmin，M 步用簇内均值更新中心，交替到中心移动量小于阈值。距离矩阵用广播一次算出来：`(X[:,None,:] - C[None,:,:])**2` 再沿最后一维求和，得到 N×k 的矩阵。要处理空簇，工程上还要多次初始化取 inertia 最小的。"

代码：

```python
def kmeans(X, k, iters=100, tol=1e-4, seed=0, n_init=1):
    X = np.asarray(X, dtype=np.float64)
    rng = np.random.default_rng(seed)
    best = (None, None, np.inf)
    for _ in range(n_init):
        centers = X[rng.choice(len(X), k, replace=False)].copy()   # 随机选 k 个样本当中心
        labels = np.zeros(len(X), dtype=int)
        for _ in range(iters):
            d = ((X[:, None, :] - centers[None, :, :]) ** 2).sum(axis=-1)   # [N,k]
            labels = d.argmin(axis=1)                                       # [N]
            new_centers = centers.copy()
            for j in range(k):
                mask = labels == j                  # 布尔数组 [N]
                if mask.any():                      # 空簇保留旧中心
                    new_centers[j] = X[mask].mean(axis=0)
            shift = np.linalg.norm(new_centers - centers)
            centers = new_centers
            if shift < tol:
                break
        inertia = ((X - centers[labels]) ** 2).sum()
        if inertia < best[2]:
            best = (labels, centers, inertia)
    return best[0], best[1]
```

逐行讲解（这题是 numpy 的"考试题"）：

① 距离矩阵那一行，拆开看：

```python
X[:, None, :]                           # (N,d) -> (N,1,d)      给每个样本插一个"中心"维
centers[None, :, :]                     # (k,d) -> (1,k,d)      给每个中心插一个"样本"维
X[:, None, :] - centers[None, :, :]     # 广播 -> (N,k,d)       每个样本减每个中心
(...) ** 2                              # 逐元素平方
.sum(axis=-1)                           # (N,k,d) 沿 d 求和 -> (N,k)   ← 距离平方矩阵
```

- 语法点 1：**索引里的 `None` 是"插一个新轴"**（等价 `np.newaxis`）。C++ 里你没法这么干，这是 numpy 独有的表达方式。
- 语法点 2：**广播让 (N,1,d) 和 (1,k,d) 变成 (N,k,d)**——规则见 2.3 节。
- 语法点 3：`.sum(axis=-1)` 沿最后一维压掉，得到 N×k。**写成 `axis=1` 也对（因为 d 是第 2 维，-1 就是 1）**，但 `-1` 更通用。
- 性能提示：N 和 k 都很大时这个 (N,k,d) 中间张量吃内存，工程上用 `scipy.spatial.distance.cdist` 或分块。面试提一句"大规模会分块/用 cdist"即可。

② **`d.argmin(axis=1)`**：沿每一行找最小值的**下标**（不是值）。C++ 对照：

```cpp
for (int i = 0; i < N; ++i) { float best = 1e30f; for (int j = 0; j < k; ++j) if (d[i][j] < best) { best = d[i][j]; lab[i] = j; } }
```

③ **`centers.copy()`**：Python 的 `=` 是引用！不 copy 就会在上一步的数组上原地改，导致 `shift` 永远算成 0、提前收敛。**这是 Python 新手最典型的静默 bug**，也值得在面试里主动说一句"这里必须 deep copy"。

④ **`mask = labels == j` → 布尔数组；`X[mask]` → 取出这些样本；`.mean(axis=0)` → 沿样本维求均值**，得到新的中心 (d,)。`mask.any()` 判空簇。

⑤ **`np.linalg.norm(v)`**：向量二范数。等价 C++ 里 `sqrt(sum(v*v))`。

⑥ **`inertia = ((X - centers[labels]) ** 2).sum()`**：`centers[labels]` 是**用标签数组反查中心**（高级索引！），得到每个点所属的中心 (N,d)，再算距离平方总和。这句很 numpy，值得单独记：**`centers[labels]` 是"按标签把中心分配给每个点"**。

追问 Q&A（kmeans 的追问比代码更重要）：

- **Q：k 怎么选？** A：肘部法（inertia 随 k 的拐点）、轮廓系数 silhouette、Gap statistic。
- **Q：初始化？** A：k-means++（按"到已有中心的最近距离平方"成比例采样），或跑 `n_init` 次取 inertia 最小。

- **Q：一定收敛到全局最优吗？——不是。这条我有实测数据**： 我在三个高斯簇（150 个点）上跑 20 个随机种子，**单次初始化的结果：min=125、中位=150、max=150，也就是 20 次里有 7 次没聚对**；改成 `n_init=20` 取最优后稳定 150/150。**面试官问到"局部最优"时，直接讲这个数字。**
- **Q：复杂度？** A：O(N · k · d · iters)。加速：MiniBatch、Elkan 三角不等式、先 PCA 降维。
- **Q：假设与局限？** A：凸簇、各向同性、方差/大小相近；对异常值敏感；硬分配（软分配要 GMM）。
- **Q：numpy 里怎么避免 (N,k,d) 的内存爆炸？** A：用 `scipy.spatial.distance.cdist(X, C, 'sqeuclidean')`，或分块计算，或利用 $||x||^2 + ||c||^2 - 2x \cdot c$ 展开成两次矩阵乘。

最易错 3 点：忘 `copy()` 导致原地修改；空簇不处理（除零得 nan）；用 `argmax` 而不是 `argmin`。

3.6 Self-Attention（单头，numpy 版只要 8 行）

面试官原话："用 Python 手写 self-attention。"

30 秒口述：

> "X 乘三个权重得到 Q、K、V；Q 乘 K 转置得到 L×L 分数，除 √d 缩放；mask 位置置 -1e9；沿最后一维 softmax；最后乘 V。numpy 里矩阵乘用 `@`，转置用 `.T`。"

代码：

```python
def self_attention(X, Wq, Wk, Wv, mask=None):
    Q = X @ Wq                              # [L,D]
    K = X @ Wk
    V = X @ Wv
    dk = Q.shape[-1]                        # = D
    scores = Q @ K.T / math.sqrt(dk)        # [L,D] @ [D,L] -> [L,L]
    if mask is not None:
        scores = np.where(mask, -1e9, scores)    # 逐元素三目
    attn = softmax(scores, axis=-1)         # 沿最后一维（每个 query 对所有 key）
    return attn @ V, attn                   # [L,L] @ [L,D] -> [L,D]
```

逐行讲解：

写法	语法点	对应 C++
`X @ Wq`	`@` 是矩阵乘运算符（Python 3.5+）；等价 `np.dot(X, Wq)`	`matmul(X, Wq, L, D, D)`
`K.T`	转置（2 维就是行列互换）	索引里换位置 `K[j*D+t]`
`Q.shape[-1]`	取最后一维长度，**比硬写 D 更安全**	传参 D
`Q @ K.T / math.sqrt(dk)`	矩阵乘完再逐元素除（`/` 对标量是逐元素）	循环里 `* scale`
`np.where(mask, -1e9, scores)`	**逐元素三目**：mask 为 True 取 -1e9，否则取原值	`if (mask[i]) S[i] = -1e30f;`
`softmax(scores, axis=-1)`	复用 3.1 的函数，按行归一化	`softmaxInPlace(S, i*L, L)`
`attn @ V`	[L,L] @ [L,D]	`matmul(attn, V, L, L, D)`

注意 mask 的两种写法（面试可能问你偏好哪种）： - `np.where(mask, -1e9, scores)`：生成新数组，不改原值。 - `scores[mask] = -1e9`：**就地修改**（更省内存），但会改到调用方的数组，需要先 copy。

追问 Q&A： - **Q：为什么除 √dk？** A：q · k 的方差随 dk 线性增长，不缩放会让 softmax 饱和、梯度消失。 - **Q：复杂度？** A：时间 $O(L^2D)$、显存 $O(L^2)$。**FlashAttention 省的是显存和 HBM 读写，不减少 FLOPs**。 - **Q：`K.T` 在多 batch 下怎么写？** A：3 维要用 `K.transpose(-2, -1)`；**`.T` 只对 2 维有效**（这是踩过的坑，值得主动说）。 - **Q：mask 一行全 True 会怎样？** A：`exp(-1e9 - max)` 全是 0，分母 0 → nan。要么保证每行至少一个有效位，要么对全屏蔽行单独置 0。 - **Q：self 和 cross 的区别？** A：Q 的来源。**多模态融合就是 cross-attention：图像 token 当 K/V、文本 token 当 Q（或反之）**——这条一定要主动讲，能接上你简历里的视觉-激光融合。

最易错 3 点：`K.T` 用错（写成 `K`）；softmax 不指定 `axis=-1`；mask 用 0 表示屏蔽。

3.7 Multi-Head Attention（重点题，numpy 版 20 行）

面试官原话："改成多头。"

30 秒口述：

> "多头就是把 D 按**列**切成 h 份，每份 dk=D/h。第 i 个头取 Wq/Wk/Wv 的第 i 个列块，各自算一遍 attention，得到 h 个 L×dk，沿最后一维 concat 回 L×D，最后乘 Wo。切片用 `Wq[:, hi*dk:(hi+1)*dk]`。"

代码：

```python
def multi_head_attention(X, Wq, Wk, Wv, Wo, h, mask=None):
    L, D = X.shape
    dk = D // h
    outs, attns = [], []
    for hi in range(h):
        sl = slice(hi * dk, (hi + 1) * dk)        # 第 hi 个头占的列区间
        Q = X @ Wq[:, sl]                         # [L,D] @ [D,dk] -> [L,dk]
        K = X @ Wk[:, sl]
        V = X @ Wv[:, sl]
        scores = Q @ K.T / math.sqrt(dk)
        if mask is not None:
            scores = np.where(mask, -1e9, scores)
        a = softmax(scores, axis=-1)
        outs.append(a @ V)                        # [L,dk]
        attns.append(a)
    O = np.concatenate(outs, axis=-1)             # 沿最后一维拼 [L,D]
    return O @ Wo, np.stack(attns, axis=0)        # [h,L,L]
```

逐行讲解：

- `sl = slice(hi*dk, (hi+1)*dk)`：等价于 `hi*dk : (hi+1)*dk` 这个切片，写成 `slice` 对象是为了复用。
- `Wq[:, sl]`：**逗号左边是行（全取），右边是列（只取这一段）** → 得到 (D, dk)。这就是"按列切头"，**和 C++ 版里 `Ws[r*dk+c] = W[r*D + hi*dk + c]` 是同一件事**。
- `X @ Wq[:, sl]`：(L,D) @ (D,dk) → (L,dk)，每个头在自己的 dk 维子空间里算。
- `np.concatenate(outs, axis=-1)`：h 个 (L,dk) 沿最后一维拼 → (L,D)。**C++ 里就是 `Ocat[i*D + hi*dk + c]` 那个双重循环。**
- `np.stack(attns, axis=0)`：h 个 (L,L) 堆成 (h,L,L)。**`stack` 是"新建一维"，`concatenate` 是"在已有维上接"**——这两个函数别搞混。
- `O @ Wo`：输出投影，不能忘。

我在本地实测过的三个性质（可当"你怎么确认写对了"的答案）：1. **`h=1` 且 `Wo=单位阵` 时，多头版和单头版逐元素完全一致**（我断言里用 `np.allclose` 验证过）。2. 每个头每一行的 attention 权重和 = 1。3. 输出 shape 恒为 (L,D)。

追问 Q&A：- **Q：参数量？** A：$4D^2$（Q/K/V/O 各 D×D），**与 h 无关**。- **Q：为什么多头有用？** A：不同头在不同子空间学不同依赖（位置、句法、指代、跨模态对应）。- **Q：MQA/GQA 省什么？** A：省推理时的 **KV cache**：MHA 每 token 存 2D，MQA 存 2D/h，GQA 存 2D·g/h；训练 FLOPs 基本不变。- **Q：numpy 用列表 append 再 concat，效率会不会差？** A：h 很小（8~32），完全无所谓；真要快就一次分配 `O = np.zeros((L, D))` 再按列赋值。- **Q：PyTorch 版怎么写？** A：见第 4 章，核心差别是 `view + transpose` 代替切片循环。

最易错 3 点：切行而不是切列（应该切 `W` 的**列**）；`dk` 忘了替换进缩放；忘了乘 `Wo`。

3.8 CART 分类树（Python 里最长的一题，但递归比 C++ 好写）

面试官原话："手写决策树 / CART 怎么分裂。"

30 秒口述：

> "CART 是二叉树，用基尼不纯度。对每个特征的每对相邻取值中点试切分，选加权基尼下降最多的做二分，递归到达到停止条件。叶子取多数类。停止条件：深度上限、样本数小于 2 倍最小叶子样本、节点已纯。"

代码：

```python
class Node:
    __slots__ = ("feat", "thr", "left", "right", "value")
    def __init__(self):
        self.feat, self.thr, self.left, self.right, self.value = -1, 0.0, None, None, 0

def _gini(y, n_cls):
    if len(y) == 0:
        return 0.0
    p = np.bincount(y, minlength=n_cls) / len(y)     # 每类占比
    return 1.0 - (p ** 2).sum()

def _build(X, y, depth, max_depth, min_leaf, min_gain, n_cls):
    node = Node()
    node.value = int(np.bincount(y, minlength=n_cls).argmax())    # 叶子值 = 多数类
    if depth >= max_depth or len(np.unique(y)) == 1 or len(y) < 2 * min_leaf:
        return node
    parent = _gini(y, n_cls)
    if parent < 1e-12:
        return node
    n = len(y)
    best_gain, best = min_gain, None
    for f in range(X.shape[1]):
        vals = np.unique(X[:, f])
        thrs = (vals[:-1] + vals[1:]) / 2.0          # 相邻中点
        for t in thrs:
            m = X[:, f] <= t                          # 布尔数组：走左边
            nl = int(m.sum())
            nr = n - nl
            if nl < min_leaf or nr < min_leaf:
                continue
            g = parent - (nl * _gini(y[m], n_cls) + nr * _gini(y[~m], n_cls)) / n
            if g > best_gain:
                best_gain, best = g, (f, t, m)
    if best is None:
        return node
    f, t, m = best
    node.feat, node.thr = f, t
    node.left = _build(X[m], y[m], depth + 1, max_depth, min_leaf, min_gain, n_cls)
    node.right = _build(X[~m], y[~m], depth + 1, max_depth, min_leaf, min_gain, n_cls)
    return node
```

逐行讲解（Python 语法点为主）：

写法	语法点
`class Node:` + `def __init__(self)`	构造函数；`self.x` 是成员变量，等价 C++ 的 `this->x`
`__slots__ = (...)`	声明只有这几个属性，省内存（面试可以不提，写不写都对）
`self.feat, self.thr, ... = -1, 0.0, None, None, 0`	Python 支持**多重赋值**，一行初始化五个成员
`None`	空指针，等价 C++ 的 `nullptr`
`np.bincount(y, minlength=n_cls)`	统计每个整数类别出现几次 → 长度 n_cls 的计数数组
`.argmax()`	返回最大值的位置 = 多数类的类别号
`p = bincount / len(y)`	计数数组除样本数 → 概率（逐元素除）
`1.0 - (p ** 2).sum()`	`**` 是幂，`(p**2).sum()` 就是 Σp^2
`X[:, f]`	第 f 列（所有行）
`np.unique(...)`	去重并升序排序 → 候选切点来源
`vals[:-1]`、`vals[1:]`	切片：去掉最后一个 / 去掉第一个，两者相加除 2 得到所有中点（**`[:-1]` 这种写法在 C++ 里要额外循环，numpy 一行**）
`m = X[:, f] <= t`	布尔数组，**注意 Python 里布尔运算用 `~` 取反，不是 `!`**
`y[m]`、`y[~m]`	布尔索引取子集 / 取反后的子集
`int(m.sum())`	布尔数组求和 = True 的个数（`np.bool_` 转 int）

写法	语法点
`if best is None:`	判断 None 用 `is`，**不要用 `==`**
`f, t, m = best`	元组解包，一行拆出三个值

为什么 Python 版比 C++ 版短：C++ 要维护 `idx` 下标数组、节点池、前缀扫描；Python 直接用布尔索引切 `X[m]`、`y[m]`，列表和对象都是引用传递，递归天然简洁。**代价是慢**（每次切都复制数组），面试时补一句"生产会用分位分桶 + 前缀扫描"即可。

追问 Q&A：- **Q：CART / ID3 / C4.5 区别？** 信息增益 / 增益率 / 基尼；ID3 多叉、CART 二叉。- **Q：为什么用基尼？** 无 log，算得快；与熵的划分结果高度一致。- **Q：剪枝？** 预剪枝（本代码的三个停止条件）vs 后剪枝（代价复杂度 α，自底向上合并）。- **Q：连续特征？** 排序取中点（本代码）；大样本用直方图分桶（LightGBM 255 桶）。- **Q：特征重要性？** 加权不纯度下降累积；更可靠用置换重要性。- **Q：怎么推到 GBDT/XGBoost？** 残差/负梯度 → 二阶泰勒展开 + 正则 $\gamma T + \lambda/2 \cdot \Sigma w^2$ → 最优叶权重 $w^* = -\Sigma g/(\Sigma h+\lambda)$ → 直方图并行。LightGBM 是 leaf-wise + GOSS + EFB。- **Q：`np.unique` 每个特征都调一次，会不会慢？** 会（O(n log n) · d）；工程上排序一次、或用直方图；面试主动提"可以排序+前缀扫描避免重复统计"。

最易错 3 点：漏停止条件（递归爆栈）；阈值用取值本身而不是中点；`y[~m]` 写成 `y[!m]`（Python 语法错）。

---

4. PyTorch 版（面试官说"用框架写"时）

> 为什么还要准备这一版：多模态岗的团队大概率天天写 torch，面试官很可能直接说"用 PyTorch 写一下吧"。 好消息：**你已经会 numpy 版了，torch 版只差 8 个语法差异**。

4.1 numpy → torch 的 8 个差异（背完就能改）

#	numpy	torch	说明
1	`x.max(axis=-1, keepdims=True)`	`x.max(dim=-1, keepdim=True).values`	torch 的 `max` 返回 (values, indices)，**必须取 `.values`**；参数名是 `dim`、`keepdim`（没有 s）
2	`x.sum(-1)`	`x.sum(-1)` 或 `x.sum(dim=-1)`	通用
3	`np.where(mask, -1e9, s)`	`s.masked_fill(mask, float('-inf'))`	torch 用 `masked_fill` 就地填；注意**语义：True = 屏蔽**
4	`logits[np.arange(N), t]`	`logits.gather(1, t[:, None]).squeeze(1)`	torch 没有高级索引，用 `gather` 按**维度**取
5	`.reshape(...)`	`.view(...)` / `.reshape(...)`	**`view` 要求内存连续**，transpose 之后必须先 `.contiguous()`
6	`x.T`	`x.transpose(-2, -1)`	torch 的 `.T` 只对 2 维且已弃用，统一用 `transpose`
7	标量就是 float	要 `.item()`	打印或比较时用 `x.item()` 取值
8	无梯度	`loss.backward()` + `p.grad`	手撕一般不用，但"能接进训练"是加分点

最大的一个坑（面试官爱看）：`view` 和 `contiguous` 的关系。

```
x = torch.randn(2, 4, 8)
y = x.transpose(1, 2)        # [2,8,4]，但内存不连续
y.view(2, 32)                # ❌ RuntimeError: view size is not compatible
y.contiguous().view(2, 32)   # ✅ 先复制成连续内存，再 view
y.reshape(2, 32)             # ✅ reshape 内部会自动处理（但可能悄悄复制）
```

讲法："拆头和拼头都要经过 transpose，所以拼头时必须 `contiguous()`，否则 `view` 报错。"

4.2 完整 MHA（已实测通过，与官方实现逐元素一致）

```
import math, torch, torch.nn as nn

def mha_core(Q, K, V, h, mask=None):
    """Q/K/V: [B,L,D], mask: 可广播到 [B,h,Lq,Lk] 的 bool，True = 屏蔽。"""
    B, Lq, D = Q.shape
```

```
    Lk = K.size(1)
    dk = D // h
    Qh = Q.view(B, Lq, h, dk).transpose(1, 2)      # [B,h,Lq,dk]
    Kh = K.view(B, Lk, h, dk).transpose(1, 2)
    Vh = V.view(B, Lk, h, dk).transpose(1, 2)
    s = Qh @ Kh.transpose(-2, -1) / math.sqrt(dk)  # [B,h,Lq,Lk]
    if mask is not None:
        s = s.masked_fill(mask, float('-inf'))
    a = torch.softmax(s, dim=-1)
    o = (a @ Vh).transpose(1, 2).contiguous().view(B, Lq, D)   # 拼头
    return o, a

class MHA(nn.Module):
    def __init__(self, d_model, n_head):
        super().__init__()
        assert d_model % n_head == 0
        self.h = n_head
        self.wq = nn.Linear(d_model, d_model)
        self.wk = nn.Linear(d_model, d_model)
        self.wv = nn.Linear(d_model, d_model)
        self.wo = nn.Linear(d_model, d_model)

    def forward(self, q, k, v, mask=None):
        out, attn = mha_core(self.wq(q), self.wk(k), self.wv(v), self.h, mask)
        return self.wo(out), attn
```

与 numpy 版的对应关系（面试时可以这么讲）：

numpy 版	torch 版	含义
`Wq[:, sl]` 切片循环	`view(B,L,h,dk).transpose(1,2)`	都是"把 D 切成 h 个 dk"
`np.concatenate(outs, -1)`	`.transpose(1,2).contiguous().view(B,L,D)`	都是"拼回 D"
`np.where(mask, -1e9, s)`	`s.masked_fill(mask, -inf)`	都是"屏蔽"
手动 `softmax(s, axis=-1)`	`torch.softmax(s, dim=-1)`	同

4.3 torch 版 softmax + 交叉熵（与 F 一致，已实测）

```
def torch_softmax(x, dim=-1):
    x = x - x.max(dim=dim, keepdim=True).values
    e = x.exp()
    return e / e.sum(dim=dim, keepdim=True)

def torch_cross_entropy(logits, target, weight=None):
    m = logits.max(dim=1, keepdim=True).values
    lse = m.squeeze(1) + torch.log(torch.exp(logits - m).sum(dim=1))
    nll = lse - logits.gather(1, target[:, None]).squeeze(1)
    if weight is not None:
        w = weight[target]
        return (nll * w).sum() / w.sum()
    return nll.mean()
```

实测结果：`torch_softmax` 与 `F.softmax` 一致；`torch_cross_entropy` 与 `F.cross_entropy` 一致（**含 `weight` 参数时也一致**）。

4.4 torch 版 kmeans（用 cdist 替代手写距离矩阵）

```
def torch_kmeans(X, k, centers0, iters=100):
    X = torch.as_tensor(X, dtype=torch.float32)
    C = torch.as_tensor(centers0, dtype=torch.float32).clone()
    for _ in range(iters):
        d = torch.cdist(X, C)                  # [N,k] 成对距离，省掉 (N,k,d) 中间张量
        lab = d.argmin(dim=1)
        newC = C.clone()
        for j in range(k):
            m = lab == j
            if m.any():
                newC[j] = X[m].mean(dim=0)
        if torch.allclose(newC, C, atol=1e-5):
            break
        C = newC
    return lab
```

实测：在同一初始中心下，torch 版与 numpy 版的分配结果**逐点完全一致**。

---

5. 怎么知道自己写对了（面试最值钱的技能）

面试官常问："你怎么确认这个实现是对的？" 大部分人说"我跑了一下没报错"。你可以给三档答案：

5.1 第一档：数学性质自检（不需要参照物）

题	必查性质
softmax	每行和为 1；对加常数不变（平移不变性）；大数（1e3）不出 nan
交叉熵	等于 `-log(softmax(x)[target])`；恒为正
LayerNorm	输出每个样本 mean≈0、var≈1
MHA	每个头每行 attention 和为 1；输出 shape 与输入一致；mask 位置权重为 0
kmeans	inertia 单调不增；多次初始化后 inertia 稳定
CART	训练集能拟合到 100%（过拟合是树的本性，正好用来验证实现没写错）

5.2 第二档：拿官方实现当参照物（最硬的答案）

```
# torch 版 attention 的正确性，直接和官方 SDPA 对拍
ref = F.scaled_dot_product_attention(Qh, Kh, Vh)                    # 无 mask
ref_causal = F.scaled_dot_product_attention(Qh, Kh, Vh, is_causal=True)
# 交叉熵对拍
F.cross_entropy(logits, target)
```

注意一个大坑：`F.scaled_dot_product_attention(attn_mask=...)` 的 **bool 语义是 `True = 允许参与`**，和我们手写 `masked_fill(mask, -inf)`（`True = 屏蔽`）**正好相反**。面试时主动提这一点，会显得你真的用过。

我的实测：无 mask 和因果 mask 两种情况，手写版与 SDPA **逐元素一致（1e-5 内）**。

5.3 第三档：退化测试（特殊输入下应退化成简单形式）

- 多头 attention 设 `h=1` 且 `Wo=单位阵` → 应与单头完全一致（**我实测一致**）。
- softmax 输入是 one-hot 的大值 → 输出应几乎还是 one-hot。
- kmeans 设 `k=1` → 所有点一个标签，中心 = 全局均值。

面试话术：

> "我一般做三层验证：先查数学性质（比如 attention 每行和为 1），再用官方算子对拍（SDPA / F.cross_entropy），最后做退化测试（h=1 应该退化成单头）。"

这段话能让面试官对你的工程严谨性直接上一个档。

---

6. 练习计划（针对 Python 弱的 3 天版）

时间	任务	验收
D1 上午	第 1、2 章（语法 + numpy 五个概念）	能不看文档写出：按行 softmax、按列求和、`X[:,None,:]` 的含义、布尔索引取子集
D1 下午	3.1 softmax + 3.2 交叉熵（numpy）	不看书写出；能说 `keepdims` 为什么必须、高级索引怎么取 target
D2 上午	3.5 kmeans（numpy）	能默写距离矩阵那一行并解释广播；能讲局部最优的实测数字（20 次里 7 次失败）

时间	任务	验收
D2 下午	3.6 单头 → 3.7 多头（numpy）	单头 10 分钟写完；多头只改 4 处（切片、dk、concat、Wo）
D3 上午	3.8 CART（numpy）	二分类训练集 100%；能讲基尼/停止条件/GBDT 推导
D3 下午	第 4 章 torch 版 + 第 5 章验证法	能把 numpy 版 10 分钟内改成 torch 版（`view/transpose/contiguous/masked_fill/gather` 五个词不出错）

面试前一晚自检清单（全打勾再睡）

- [ ] `keepdims=True` / `keepdim=True` 什么时候必须加
- [ ] `axis=-1` 是"最后一维"，softmax/LN 都用它
- [ ] 广播三规则，以及 `X[:, None, :]` 是在中间插一个轴
- [ ] `logits[np.arange(N), target]` 是交叉索引取正确类
- [ ] `argsort(-scores)` 是降序下标（NMS 必用）
- [ ] `center[labels]` 是"按标签分配中心"（kmeans inertia 那句）
- [ ] `copy()` 必须有，Python 赋值是引用
- [ ] `@` 矩阵乘、`.T` 转置（3 维要用 `transpose(-2,-1)`）
- [ ] torch 的 `max(...).values`、`gather`、`masked_fill`、`contiguous().view`
- [ ] SDPA 的 bool mask 语义与 `masked_fill` **相反**
- [ ] 多层验证三件套：数学性质 → 官方对拍 → 退化测试

附录：配套文件与实测结果

文件：`ai_interview_py.py`（numpy + torch，与本文档代码一致）

```
python3 ai_interview_py.py       # 需要 numpy；装了 torch 会自动跑 torch 部分
```

本机实测（python3 + numpy + torch 2.14.1+cpu）：39 项断言全部通过，0 失败

模块	断言数	关键结论
softmax / log_softmax	5	行和为 1；1e3 级不溢出；平移不变；与手算一致；log_softmax 取 exp 可还原
cross_entropy	3	与按定义手算一致；带 weight 可跑；1e3 级 logits 不溢出
layer_norm	1	每个样本 mean≈0、var≈1
IoU / NMS	3	IoU=81/119；无交集为 0；NMS 保留
kmeans	2	**单次初始化 20 个 seed：min=125、中位=150、max=150，7/20 没聚对**；`n_init=20` 后稳定 150/150
self-attention	5	行和=1；输出=A·V；因果 mask 上三角=0；mask 后行和仍=1；下三角比例保持
multi-head	4	形状 [h,L,L]/[L,D]；每头行和=1；**h=1 且 Wo=I 时与单头逐元素一致**
CART	3	二分类/三分类 ACC=1.0；根阈值落在簇间（thr=1.523）
torch softmax/CE	3	与 `F.softmax`、`F.cross_entropy`（含 weight）一致
torch MHA	7	与 `F.scaled_dot_product_attention`（无 mask / 因果 mask）逐元素一致；mask/padding 位置权重为 0；行和=1
torch nn.Module	2	前向形状正确；反向传播有梯度

模块	断言数	关键结论
torch kmeans	1	与 numpy 版在同初始中心下分配完全一致

怎么用这份文件练：先关掉文件在纸上写 → 打开对照 → 删掉重写 → 跑 `python3 ai_interview_py.py` 看断言是否全绿。